The Journal of Open Source Software

# Mamma Mia: A Python library for simulating underwater platform payloads using ocean model data

**Thomas Prime** [1]

**1** National Oceanography Centre, Southampton, United Kingdom

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:** Open Journals

**Reviewers:**

- @openjournals

**Submitted:** 01 January 1970
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

## Summary

Autonomous underwater vehicles (AUVs) and ocean gliders rely on a wide variety of onboard payloads—such as CTDs, ADCPs, fluorometers, and passive acoustic sensors—to collect high-resolution in situ ocean data. Before deployment, mission planners and researchers often need to simulate how these payloads will behave when embedded in complex 3D or 4D ocean environments. This includes anticipating sampling resolution, environmental conditions, and sensor-specific responses to varying water masses. Mamma Mia is an open-source Python library designed to address this need. It provides a flexible framework for simulating the behaviour of glider and AUV payloads within model-derived ocean fields, using data from online sources such as the National Oceanography Centre and the Copernicus Marine Service or locally from downloaded model data. The library supports multiple platforms at once and allows users to simulate campaigns with a diverse range of platforms. Optionally the library can also simulate the behaviour of specific platform types, Slocum gliders and Autosub's. The library has been developed with an emphasis on usability, extensibility, and performance. It is built on widely used scientific Python libraries—NumPy, Pyinterp, Xarray, and Zarr—and can be integrated into real-time or offline mission planning workflows. It supports exporting to Zarr, a format suitable for visualization tools such as ParaView.

## Statement of Need

Ocean scientists often rely on numerical ocean models to plan missions and interpret observational data. Yet, while many community tools exist for working with model outputs, there is a lack of streamlined, open-source software aimed specifically at simulating sensor payloads on mobile marine platforms. Researchers currently rely on ad-hoc scripts, isolated institutional software, or manual workflows to: * Interpolate 4D model fields onto observed or simulated glider trajectories * Apply realistic sensor sampling behaviour (e.g., burst sampling, averaging, noise models) * Estimate expected data volumes and environmental variability along planned missions * Combine multiple payloads into a single simulated dataset

Mamma Mia fills this gap by providing an accessible, well-tested, well-documented Python toolkit for payload simulation using model data. It enables researchers, engineers, and students to test mission strategies, evaluate scientific return, and develop improved onboard algorithms, without requiring access to the physical vehicles.

## Features

- 4D interpolation engine supporting temperature, salinity, currents, biogeochemical tracers, and more
- Support for glider-sawtooth trajectories or arbitrary paths
- Built-in payloads:

DRAFT

  - CTD sampling with noise and response-time emulation
  - Radiometers (Chlorophyll and Downwelling Radiative Flux)
  - Dissolved Gas (Oxygen)
- Able to create custom sensors, platforms and parameters
- Model input formats:
  - CMEMS (Copernicus marine toolbox)
  - NOC (OceanDataStore)
    * Local NetCDF/Zarr
- Trajectory input formats
  - CSV waypoints
  - NetCDF with trajectory (e.g. from real deployment)
- Simulated platform (Slocum gliders and Autosubs)
- Export formats:
  - Zarr

## Example usage

```python
from mamma_mia import (
    add_mission,
    create_campaign,
    create_interpolator,
    create_mission,
    create_platform,
    create_trajectory,
    fly,
    get_data,
    plot_path,
    start_payload_dashboard,
)

spec_file = "spec_files/glider_spec_virtual_mooring.toml"
traj = create_trajectory(spec_file=spec_file)
platform = create_platform(spec_file=spec_file)

# create mission using trajectory and platform datasets
# a payload dataset is created but is currently empty
mission = create_mission(
    mission_name="Example Glider RAPID",
    summary="Virtual glider performing mooring replacement mission at RAPID",
    platform=platform,
    trajectory=traj,
    apply_obs_error=True,
)
# get data from specified souce in spec file
# note mission is returned with locations of data stored as attributes
mission = get_data(mission=mission)

# create interpolators from downloaded datasets
interpolator = create_interpolator(mission=mission)

# fly the mission by interpolating the downloaded data onto the payload dataset
mission = fly(mission=mission, interpolators=interpolator)

# create a campaign to store the mission (missions can be standalone)
```

DRAFT

```
campaign = create_campaign(
    campaign_name="RAPID virtual mooring",
    description="single glider performing virtual mooring",
)
campaign = add_mission(campaign=campaign, mission=mission)
# export to zarr (netcdf should be possible too)
campaign.to_zarr("RAPID.zarr", "w", consolidated=False)
# simple plot to show payload path
plot_path(missions=mission)
# plot a mission payload
start_payload_dashboard(missions=mission)
```

## Figures

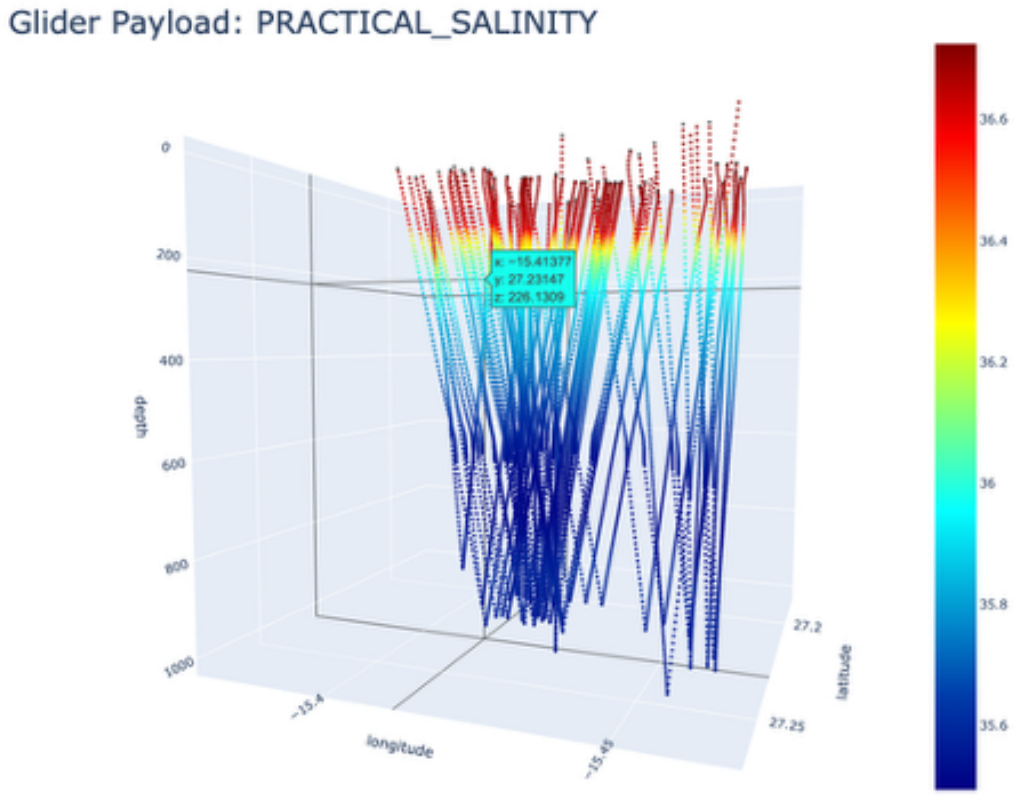



**Figure 1:** output_example

## Community Guidelines

Mamma Mia welcomes contributions through pull requests, GitHub issues, and community discussions.

## Acknowledgements

We acknowledge the developers of Xarray, NumPy, PyInterp and glidersim without which Mamma Mia wouldn't be possible.

## References

DRAFT